



PortaldoSudo

FUNCCIONALIDADE DO PORTAL DO MUNDO · OTIMIZAÇÃO AUTOMÁTICA DE IMAGENS

Imagens em KB

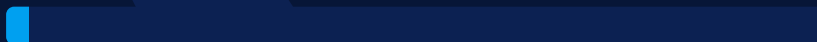
Como o Portal do Mundo transforma fotos de vários megabytes em imagens WebP de poucos quilobytes, e como levar a mesma técnica a qualquer sistema.

Foto enviada



4,87 MB

Guardada no site



141 KB

Barras à escala · upload real de 1 de outubro de 2026

AUTOR

Humberto Rodrigues

EDIÇÃO

1.^a - outubro de 2026

SISTEMA

Portal do Mundo 3.0



TÍTULO	Imagens em KB
SUBTÍTULO	Otimização automática de imagens no Portal do Mundo, e como replicá-la noutros sistemas
AUTOR	Humberto Rodrigues
SISTEMA	Portal do Mundo (www.portaldomundo.com), versão 3.0
EDIÇÃO	1.ª edição, outubro de 2026
CÓDIGO DE REFERÊNCIA	<code>private/app/Helpers/Image.php</code> , <code>Upload.php</code> , <code>MediaOptimizer.php</code>
PÚBLICO	Programadores e equipas técnicas que queiram aplicar a mesma técnica nos seus sistemas

© 2026 Humberto Rodrigues · Portal do Mundo
Documento técnico do Portal do Mundo.

Índice

1	Contextualização: porque decidi fazer isto	4
2	Vantagens e desvantagens	6
3	Medição real	8
4	Conceitos: o que torna uma imagem pesada	9
5	Formatos: JPEG, PNG, WebP, AVIF	10
6	Decisões: largura, alvo em KB, variantes	11
7	O pipeline do Portal, passo a passo	11
8	O sistema em ação: capturas do painel	13
9	Implementação em PHP (GD, Imagick, Laravel)	17
10	Implementação noutras linguagens	20
11	Compressão no browser antes do upload	24
12	Servir as imagens	24
13	Requisitos do servidor	26
14	Cuidados e erros comuns	26
15	Checklist de implementação	27

1 Contextualização

Porque decidi implementar isto no Portal do Mundo, e porque acho que outros sistemas o devem fazer.

O Portal e quem o lê

Criei o Portal do Mundo para dar espaço a quem escreve em português, com foco em Moçambique: notícias, crónicas, cultura, opinião. Qualquer criador pode publicar e ser apoiado pelos leitores através de gorjetas pagas com M-Pesa e e-Mola. Os nossos leitores e criadores usam sobretudo o telemóvel, muitas vezes com dados móveis pagos ao megabyte e com ligações que oscilam ao longo do dia.

O problema que vi

Os criadores publicam fotos tiradas diretamente no telemóvel. Cada uma tem facilmente entre 3 e 5 MB; a foto usada como exemplo neste livro tinha 4,87 MB. Isso tinha três consequências:

- **As páginas abriam partidas.** Em ligações lentas, o texto e o layout apareciam desarrumados durante vários segundos enquanto as imagens pesadas carregavam. Para quem tinha pouca rede, a primeira impressão do Portal era uma página partida.
- **Ler um artigo gastava dados.** Uma única foto de destaque pesava mais do que todo o texto, CSS e código da página juntos. Para quem paga dados ao megabyte, isso é um custo real só para ler uma notícia.
- **O espaço no servidor acabava.** O alojamento partilhado onde o Portal começou tinha armazenamento limitado. Com cada criador a enviar fotos de vários MB, o disco e os backups cresciam muito mais depressa do que o conteúdo.

A decisão

Na versão 3.0 do Portal (julho de 2026) defini uma regra simples: **todas as imagens passam a viver em KB, nunca em MB.** E decidi que isso tinha de acontecer **sozinho**. Não podia depender de cada criador saber o que é compressão, resolução ou WebP. Quem escreve deve preocupar-se com o texto; o sistema garante que a imagem chega leve ao leitor.

Foi assim que nasceu o pipeline descrito neste livro: cada foto enviada é convertida para WebP, reduzida para a largura que um ecrã realmente usa e comprimida até caber num orçamento de tamanho. As imagens antigas passaram a ter versões WebP servidas no mesmo endereço, sem partir nenhum artigo publicado.

Porque outros sistemas devem fazer o mesmo

- **A realidade dos nossos leitores não é exceção.** Em Moçambique, em grande parte de África e em muitos outros mercados, a internet chega sobretudo pelo telemóvel e pelos dados móveis. Um site pensado para fibra ótica falha para a maioria destas pessoas.
- **A velocidade conta para o Google.** O tempo de carregamento é um dos sinais que os motores de busca usam para ordenar resultados. Imagens leves ajudam o site a aparecer e a ser lido.

- **Poupa dinheiro.** Menos disco, menos tráfego e backups mais pequenos permitem que o mesmo servidor aguente mais visitas e mais conteúdo, sem mudar de plano.
- **É uma questão de respeito.** Cada megabyte desnecessário é dinheiro e tempo que o leitor gasta sem receber nada em troca.
- **Dá pouco trabalho.** A técnica usa bibliotecas gratuitas que existem em todas as linguagens principais. Este livro traz o código pronto para PHP, Node.js, Python, C#, Go, Ruby e Java.

Humberto Rodrigues

Autor do Portal do Mundo

2 Vantagens e desvantagens

Nenhuma técnica é grátis. Antes de implementar, convém saber o que se ganha e o que se perde.

Vantagens

- **Páginas mais rápidas.** A foto de destaque fica cerca de 35 vezes mais leve.
- **Menos dados para o leitor.** Ler um artigo deixa de custar megabytes.
- **Melhor posição no Google,** porque a velocidade conta na ordenação.
- **Menos disco e backups pequenos.** O backup completo do site cabe em 194 MB.
- **Menos tráfego no servidor,** por isso o mesmo plano aguenta mais visitas.
- **Privacidade.** As coordenadas GPS das fotos são removidas antes de publicar.
- **Automático e previsível.** O criador não precisa de saber nada, e todas as imagens ficam com peso semelhante.
- **Um tamanho para cada sítio.** As variantes fazem cada página carregar só a imagem de que precisa.
- **Controlo de uso.** Cada imagem mostra se está em uso e onde. Imagens em uso não podem ser apagadas por engano.

Desvantagens

- **O original é apagado.** Não há forma de recuperar a foto em resolução total a partir do site.
- **Compressão com perdas.** Ótima para ecrã, inadequada para impressão, fotografia profissional ou arquivo.
- **Upload um pouco mais lento** e mais exigente em memória (256 MB recomendados), porque o servidor processa a imagem.
- **WebP não abre em todo o lado.** Alguns clientes de email (como o Outlook para Windows) e programas antigos não o mostram.
- **GIF animado** não é otimizado com o GD; fica como foi enviado.
- **Imagens antigas** precisam de um passo extra: o otimizador e uma regra no servidor web.
- **Depende do servidor.** Ao mudar de alojamento, é preciso confirmar o suporte WebP e os limites do PHP.

Como o Portal reduz as desvantagens

RISCO	O QUE O PORTAL FAZ
Perder qualidade a mais	A qualidade nunca desce abaixo de 48, e um ficheiro só é substituído se ficar mais pequeno.
Estragar imagens pequenas	Imagens abaixo da largura máxima nunca são ampliadas.
Partir artigos antigos	As imagens antigas mantêm o nome e o formato; a versão WebP é servida no mesmo URL.
GIF animado	É detetado e deixado intacto.
Servidor sem WebP	Se a conversão falhar, a imagem segue no formato original, comprimida na mesma.
Fotos de lado	A rotação do telemóvel é corrigida antes da conversão (desde outubro de 2026).

Perder o siteBackups diários encriptados, com 7 cópias.

Quando não usar esta técnica como está

- **Portfólios de fotografia e lojas com zoom:** subir a largura máxima para 2560 px e guardar o original numa pasta privada.
- **Arquivos, documentos legais ou imagens para impressão:** guardar sempre o original sem perdas e gerar a versão WebP só para mostrar no site.
- **Newsletters por email:** enviar as imagens em JPEG, por causa dos clientes de email que não mostram WebP.

3 Medição real

Upload feito no painel do Portal a 1 de outubro de 2026 (ficheiro `IMG_4162`), com as opções de produção: alvo de 350 KB e largura máxima de 1920 px. O ficheiro original de 4,87 MB foi apagado depois da conversão; no disco ficaram só estes cinco ficheiros:

FICHEIRO	USO NO SITE	TAMANHO
<code>...-img-4162.webp</code> (1920×1280)	página do artigo, partilhas	141,1 KB
<code>...-1024x615.webp</code>	destaques grandes	53,1 KB
<code>...-740x444.webp</code>	grelhas e listagens	35,5 KB
<code>...-370x222.webp</code>	cartões, artigos relacionados	14,3 KB
<code>...-150x150.webp</code>	miniaturas, biblioteca	5,4 KB
Total no disco		249,4 KB

-97%

no ficheiro principal

-95%

contando as 4 variantes

×35

menos dados por visita a um artigo

Numa ligação móvel de 1 Mbit/s, comum em Moçambique, a foto original demoraria cerca de 40 segundos a carregar; a versão de 141 KB demora pouco mais de 1 segundo.

4 Conceitos: o que torna uma imagem pesada

Píxeis

Uma imagem digital é uma grelha de píxeis. Uma foto de telemóvel de 12 megapíxeis tem $4032 \times 3024 = 12,2$ milhões de píxeis. Um ecrã de computador típico mostra uma imagem com, no máximo, 1920 px de largura; um telemóvel, cerca de 400 px (ou 1200 px físicos num ecrã de alta densidade). Tudo o que passa disso é enviado ao visitante e deitado fora pelo browser.

O peso cresce com a **área**, não com a largura: reduzir a largura para metade corta os píxeis para um quarto. Passar de 4032 px para 1920 px elimina cerca de 77% dos píxeis antes de qualquer compressão.

Compressão com perdas e sem perdas

- **Sem perdas** (PNG, WebP lossless): o ficheiro descomprimido é idêntico ao original, bit a bit. Ótimo para logótipos, ícones e capturas de ecrã com texto; mau para fotografias, que ficam enormes.
- **Com perdas** (JPEG, WebP, AVIF): o codificador descarta detalhes que o olho humano não nota (variações finas de cor, ruído de alta frequência). É o que permite reduzir uma foto 10 a 30 vezes.

Qualidade

A "qualidade" (0 a 100) controla quanto detalhe se descarta. A relação não é linear: os últimos pontos de qualidade são os mais caros. Descer de 95–100 (o que as câmaras usam) para cerca de 80 costuma reduzir o ficheiro para metade ou menos, sem diferença visível. Abaixo de cerca de 50 começam a aparecer artefactos (blocos, contornos borrados). Por isso o Portal começa em 78 e nunca desce abaixo de 48.

Metadados

As fotos de telemóvel trazem EXIF: modelo da câmara, data, **coordenadas GPS** e uma miniatura embutida. Remover isto poupa alguns KB e, mais importante, evita publicar a localização de quem tirou a foto.

Memória no servidor

Para processar uma imagem, a biblioteca descomprime-a para RAM a 4 bytes por píxel (vermelho, verde, azul, transparência). Uma foto de 12 MP ocupa $4032 \times 3024 \times 4 \approx$ **49 MB** de memória, mesmo que o ficheiro tenha só 3 MB. Com cópias intermédias, um `memory_limit` de 128 MB pode não chegar; recomenda-se 256 MB.

5 Formatos: JPEG, PNG, WebP, AVIF

FORMATO	MELHOR PARA	TAMANHO RELATIVO (FOTO)	TRANSPARÊNCIA	SUORTE
JPEG	fotos (formato antigo)	100%	não	universal
PNG	logótipos, ícones, texto	300–500%	sim	universal
WebP	fotos e gráficos na web	65–75%	sim	todos os browsers atuais (desde 2020)
AVIF	fotos, máxima compressão	50–60%	sim	browsers atuais; alguns dispositivos antigos não

Porque WebP e não AVIF? O AVIF comprime mais 20 a 30%, mas é 5 a 10 vezes mais lento a codificar (no GD do PHP, uma foto grande pode levar vários segundos), e o suporte em telemóveis antigos ainda falha. O WebP é o melhor equilíbrio hoje: rápido, suportado em todo o lado e muito mais leve do que JPEG. Para um projeto novo com libvips ou sharp, AVIF é uma opção a considerar como segundo formato.

6 Decisões: largura, alvo em KB, variantes

Largura máxima

LARGURA	QUANDO USAR
1600 px	blogs e notícias com coluna de texto estreita
1920 px	imagem de destaque a toda a largura em ecrãs Full HD (o valor do Portal)
2560 px	fotografia, portfólios, ecrãs de alta densidade

Alvo em KB

O alvo é um **orçamento**, não uma qualidade fixa. Uma foto simples (céu, parede) fica abaixo do alvo logo à primeira; uma foto muito detalhada (folhagem, multidão) precisa de baixar a qualidade. Assim, todas as imagens ficam com peso previsível. 350 KB para 1920 px é folgado: na prática a maioria fica entre 100 e 200 KB.

Variantes

Cada sítio do layout deve carregar a imagem no tamanho em que é mostrada. Uma miniatura de 150 px que carrega a imagem de 1920 px desperdiça mais de 95% dos dados. Escolhe as variantes a partir do teu layout, não ao contrário:

VARIANTE	PROPORÇÃO	ONDE APARECE NO PORTAL
150×150	1:1	miniaturas, biblioteca de ficheiros
370×222	5:3	cartões, artigos relacionados
740×444	5:3	grelhas da página inicial (370 px em ecrã 2×)
1024×615	5:3	destaques grandes

As variantes são **recortadas ao centro** (como `object-fit: cover` em CSS): a imagem é escalada até cobrir o retângulo e o excesso é cortado, para nunca haver faixas vazias nem imagens esticadas.

7 O pipeline do Portal, passo a passo

Ordem pela qual cada upload é processado (`private/app/Helpers/Upload.php` e `Image.php`):

- 1 Validar o tamanho no browser e no servidor.** Limite `fileupload_max` (5 MB). O browser rejeita logo, sem gastar dados; o servidor valida outra vez, porque o browser pode ser contornado.
- 2 Detetar duplicados por SHA-1.** Se o mesmo utilizador enviar o mesmo ficheiro, reutiliza o registo existente em vez de guardar outra cópia e outras 4 variantes.

- 3 Converter para WebP.** JPG, PNG, BMP, AVIF e GIF estático, com qualidade 80. O GIF animado fica intacto, porque o GD não gera WebP animado. Se falhar, segue com o formato original. O ficheiro original é apagado.

- 4 Reduzir para a largura máxima.** `image_max_width` = 1920 px, mantendo a proporção. Nunca aumenta imagens pequenas.

- 5 Comprimir até caber no alvo.** Experimenta as qualidades 78, 70, 62, 55 e 48 até o ficheiro ficar abaixo de `image_target_kb` (350 KB). Só substitui o ficheiro se o resultado for menor, por isso voltar a correr nunca o aumenta.

- 6 Gerar as 4 variantes,** com recorte centrado e qualidade 78.

- 7 Guardar os metadados** (largura, altura, tamanho, SHA-1) na tabela `files`. O nome do ficheiro fica em ASCII (`time()-slug.webp`), sem acentos nem espaços, para os URLs funcionarem em qualquer servidor.

Rotação EXIF: desde 1 de outubro de 2026 o Portal corrige a orientação das fotos de telemóvel logo após o upload, antes da conversão (`Image::fixOrientation()`).

Uma melhoria em relação ao Portal, já incluída no código deste guia: reduzir antes de codificar, para comprimir só uma vez. O Portal codifica a imagem até três vezes (conversão, redução, compressão), o que é mais lento e perde um pouco de qualidade.

8 O sistema em ação

Capturas do painel do Portal do Mundo em produção, a 1 de outubro de 2026. Mostram o que criadores e administradores veem enquanto cada imagem é enviada, processada e guardada.

8.1 O criador publica um artigo

Na página **Escrever artigo**, a imagem em destaque começa a ser enviada assim que o criador a escolhe, antes de carregar em "Publicar". Um anel mostra a percentagem real do envio, calculada pelo browser.

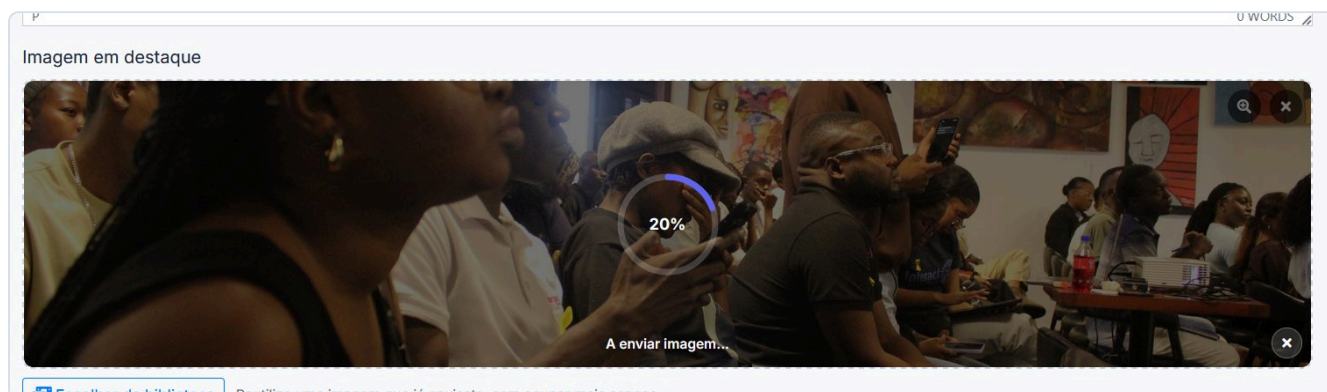


Figura 1. Envio em curso: 20%. O criador pode continuar a escrever enquanto a foto sobe.

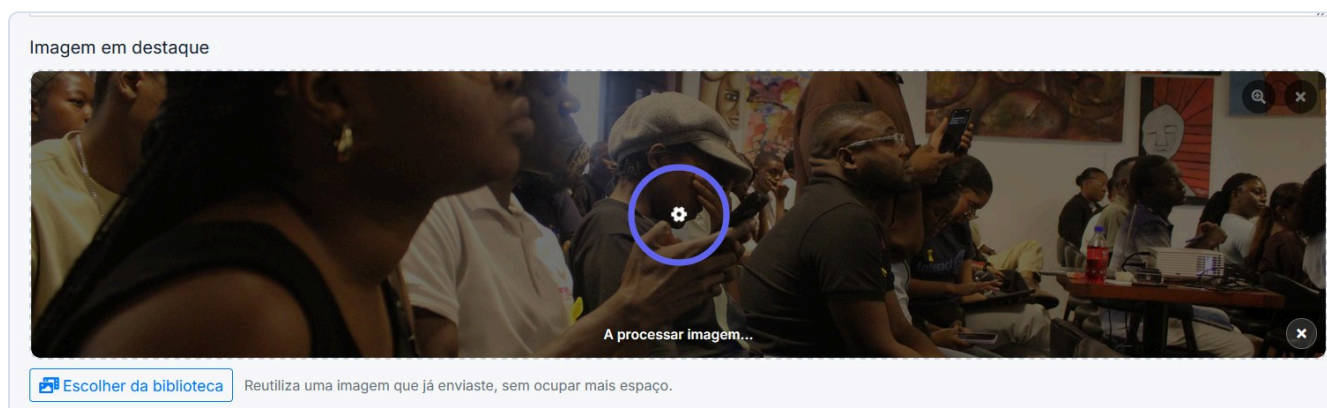


Figura 2. O envio terminou e o servidor está a converter para WebP, reduzir e comprimir. Se o criador carregar em "Publicar" nesta fase, o artigo é enviado sozinho assim que o processamento acabar.



Figura 3. Uma foto de 5,2 MB é recusada logo no browser, antes de gastar um único byte de dados móveis. O limite vem da opção `fileupload_max` do painel.

Reutilizar em vez de reenviar: o botão "Escolher da biblioteca", visível nas figuras, deixa usar uma imagem já enviada sem ocupar mais espaço. Se o mesmo ficheiro for enviado outra vez pelo mesmo utilizador, o Portal deteta-o pelo SHA-1 e reaproveita-o.

8.2 O administrador envia um ficheiro

Na área **Admin** → **Ficheiros**, qualquer formulário com upload mostra o mesmo cartão de progresso, em duas fases.

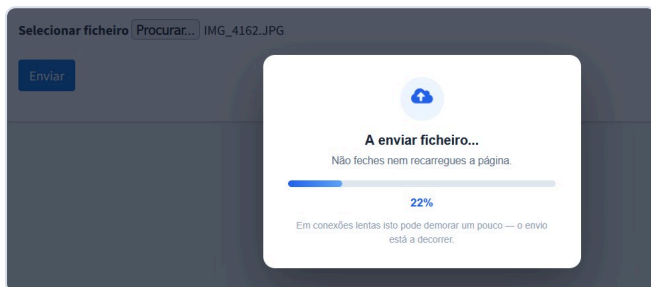


Figura 4. Envio da foto IMG_4162.JPG : 22%.

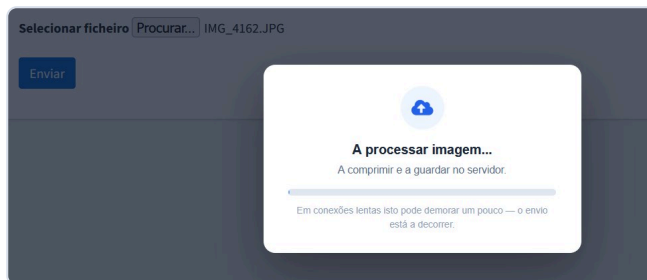


Figura 5. A foto já chegou; o servidor comprime e guarda.

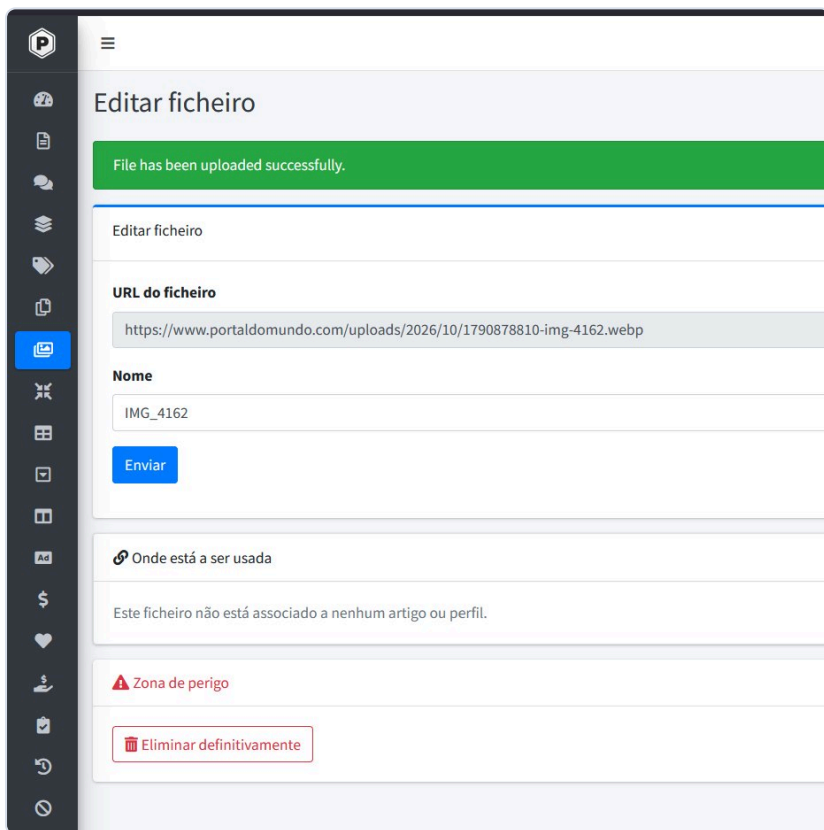


Figura 6. O resultado. Entrou um IMG_4162.JPG de 4,87 MB e ficou guardado ...-img-4162.webp, com nome em minúsculas e sem espaços. A secção "Onde está a ser usada" diz que este ficheiro ainda não está em nenhum artigo nem perfil.

8.3 Biblioteca: tamanho real e uso de cada imagem

A biblioteca mostra cada imagem com o tamanho real em disco e as dimensões. As imagens dos artigos ficam entre **19 KB** e **298 KB**, nenhuma em MB.

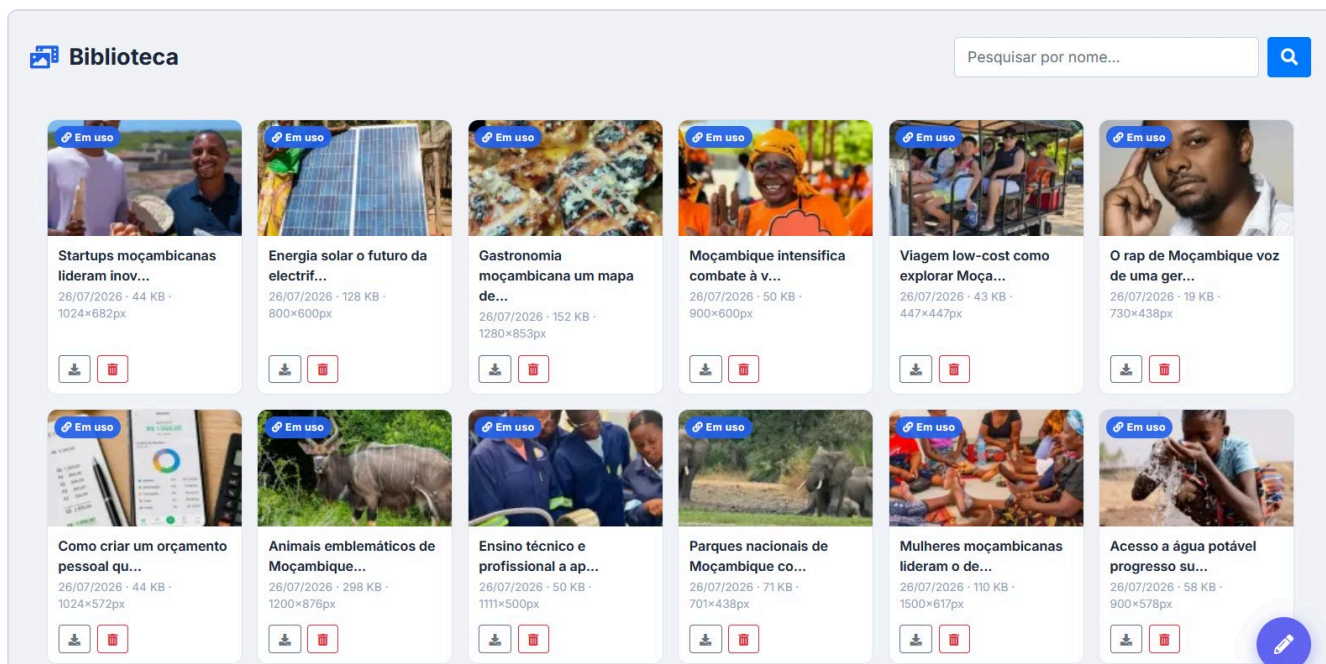


Figura 7. A biblioteca de um criador. Por exemplo: "Energia solar" com 128 KB a 800×600 px, "O rap de Moçambique" com 19 KB a 730×438 px.

Imagem em uso ou livre. O Portal identifica, para cada ficheiro, se está a ser usado e onde:

- Na grelha, o selo **"Em uso"** marca as imagens que são destaque de um artigo, foto de perfil ou capa de perfil. Ao clicar no selo vê-se a lista de artigos.
- Na página do ficheiro (Figura 6), a secção **"Onde está a ser usada"** faz a verificação completa. Inclui também os artigos que mostram a imagem **dentro do texto**.
- Uma imagem que não aparece em nenhum destes sítios está **livre** e pode ser apagada sem partir nada.
- **Proteção:** se alguém tentar apagar uma imagem em uso, o Portal recusa e diz onde ela está a ser usada ("substitui-a primeiro e depois apaga"). No painel de admin, apagar uma imagem em uso exige uma confirmação explícita.

Assim, quem gere o site pode libertar espaço com segurança: apaga só o que está livre e sabe exatamente que artigos são afetados antes de mexer numa imagem em uso.

8.4 Otimizador de imagens antigas

As imagens publicadas antes da versão 3.0 são tratadas pelo **Admin → Otimizar Imagens**. Funciona pelo browser, em lotes, sem precisar de acesso SSH ao servidor.

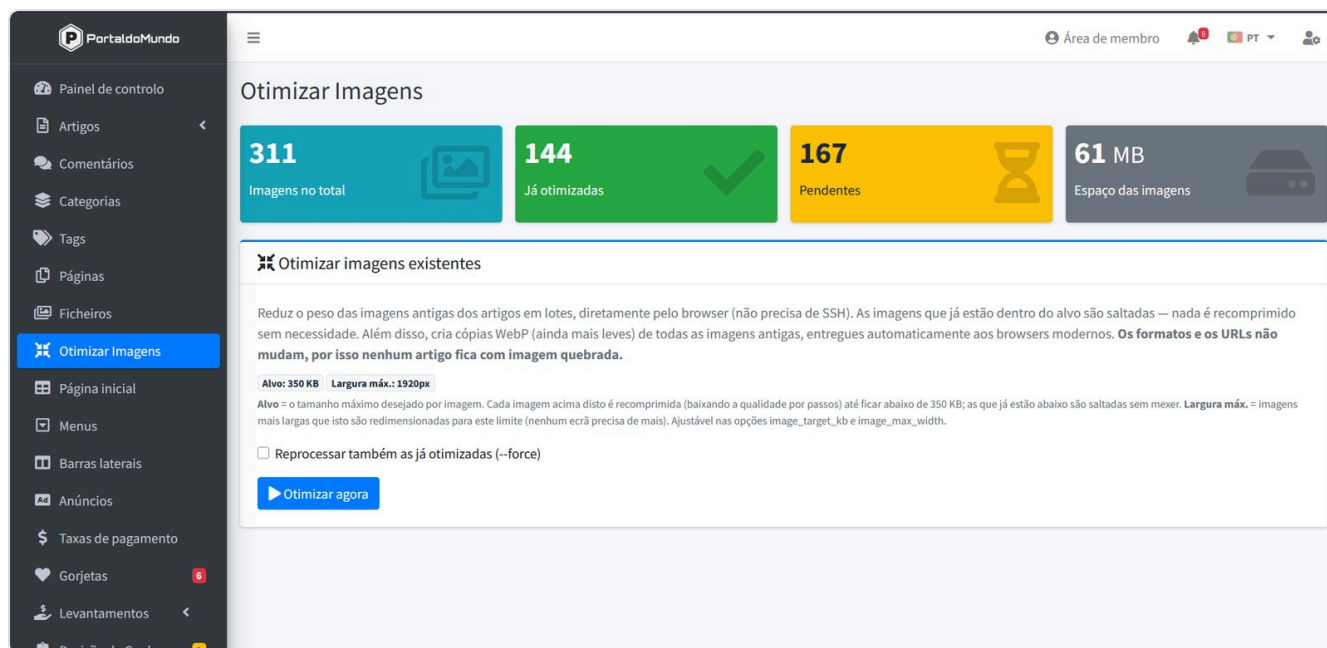


Figura 8. Antes de correr: 311 imagens no total, 144 já otimizadas, 167 pendentes, 61 MB ocupados. O alvo (350 KB) e a largura máxima (1920 px) aparecem à vista.

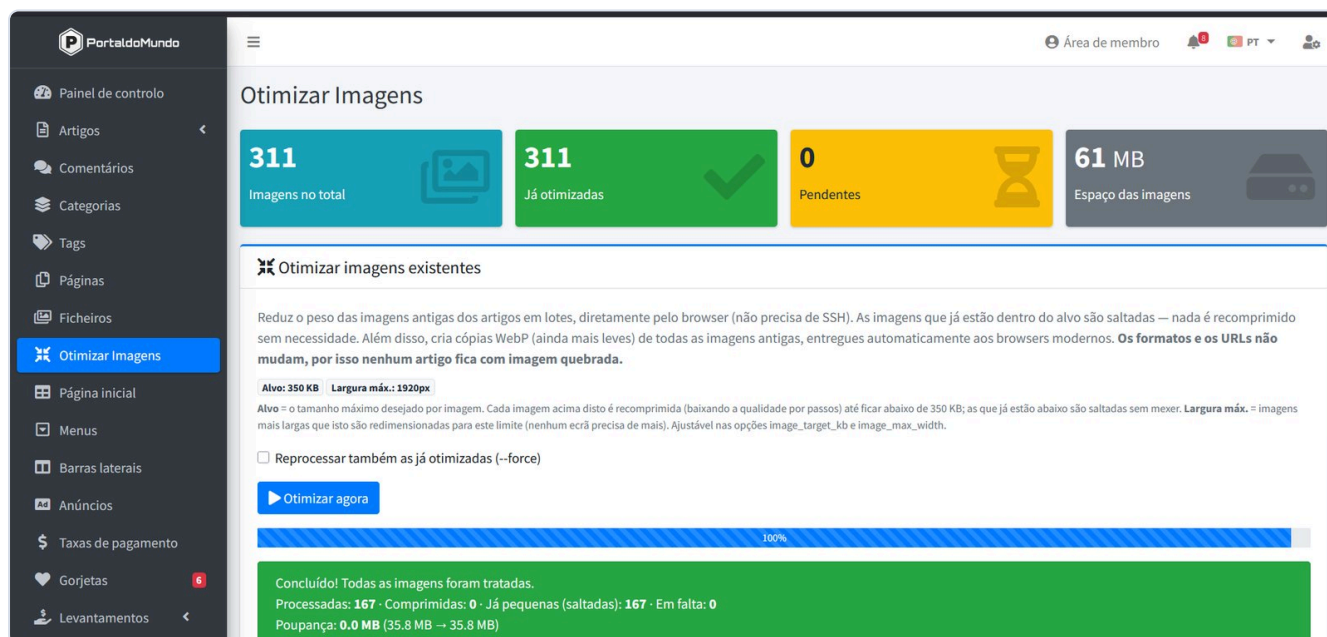


Figura 9. Depois de correr: as 167 pendentes foram verificadas. Todas já estavam abaixo do alvo de 350 KB, por isso foram **saltadas sem recomprimir** (poupança: 0,0 MB).

Porque é que a poupança foi zero? Porque o otimizador nunca recomprime uma imagem que já está dentro do alvo. Recomprimir só tiraria qualidade sem ganho real. As 167 imagens ficaram marcadas como "já pequenas", e o contador de pendentes desceu para 0. Num site com imagens antigas de vários MB, este mesmo ecrã mostraria a poupança em megabytes.

9 Implementação em PHP

9.1 PHP puro com GD

A extensão GD vem com quase todas as instalações de PHP. Este código não depende de framework e foi testado: um PNG de 2 MB ficou com 99 KB.

compress_image.php

```
<?php
/**
 * Converte para WebP, reduz para $maxWidth e baixa a qualidade até caber em $targetKb.
 */
function compress_to_webp(string $src, string $dst, int $maxWidth = 1920, int $targetKb = 350):
bool
{
    $info = @getimagesize($src);
    if (!$info) {
        return false;
    }
    [$w, $h, $type] = $info;

    $img = match ($type) {
        IMAGETYPE_JPEG => imagecreatefromjpeg($src),
        IMAGETYPE_PNG  => imagecreatefrompng($src),
        IMAGETYPE_WEBP => imagecreatefromwebp($src),
        IMAGETYPE_GIF  => imagecreatefromgif($src),    // só GIF estático
        default        => false,
    };
    if (!$img) {
        return false;
    }

    // 1. Fotos de telemóvel: o GD ignora o EXIF "Orientation" e a foto fica deitada
    if ($type === IMAGETYPE_JPEG && function_exists('exif_read_data')) {
        $orientation = @exif_read_data($src)['Orientation'] ?? 1;
        $img = match ($orientation) {
            3 => imagerotate($img, 180, 0),
            6 => imagerotate($img, -90, 0),
            8 => imagerotate($img, 90, 0),
            default => $img,
        };
        [$w, $h] = [imagesx($img), imagesy($img)];
    }

    // 2. Reduzir (nunca aumentar), mantendo a transparência
    imagepalettetotruecolor($img);
    if ($w > $maxWidth) {
        $newH = (int) round($h * $maxWidth / $w);
        $resized = imagecreatetruecolor($maxWidth, $newH);
        imagealphablending($resized, false);
        imagesavealpha($resized, true);
        imagecopyresampled($resized, $img, 0, 0, 0, 0, $maxWidth, $newH, $w, $h);
        imagedestroy($img);
        $img = $resized;
    } else {
        imagealphablending($img, false);
        imagesavealpha($img, true);
    }
}
```

```

    }

    // 3. Qualidade descendente até caber no alvo
    $tmp = $dst . '.tmp';
    foreach ([78, 70, 62, 55, 48] as $quality) {
        imagewebp($img, $tmp, $quality);
        clearstatcache(true, $tmp);
        if (filesize($tmp) <= $targetKb * 1024) {
            break;
        }
    }
    imagedestroy($img);

    return rename($tmp, $dst);
}

/** Variante com recorte centrado (equivalente a object-fit: cover). */
function make_webp_variant(string $src, string $dst, int $tw, int $th, int $quality = 78): void
{
    $img = imagecreatefromwebp($src);
    $w = imagesx($img);
    $h = imagesy($img);

    $scale = max($tw / $w, $th / $h);
    $sw = (int) round($tw / $scale);
    $sh = (int) round($th / $scale);

    $thumb = imagecreatetruecolor($tw, $th);
    imagealphablending($thumb, false);
    imagesavealpha($thumb, true);
    imagecopyresampled($thumb, $img, 0, 0, (int) (($w - $sw) / 2), (int) (($h - $sh) / 2), $tw,
    $th, $sw, $sh);
    imagewebp($thumb, $dst, $quality);

    imagedestroy($thumb);
    imagedestroy($img);
}

// Uso, depois de receber o upload:
$dir = __DIR__ . '/uploads/' . date('Y/m');
@mkdir($dir, 0755, true);
$name = time() . '-foto';
compress_to_webp($_FILES['foto']['tmp_name'], "$dir/$name.webp");
foreach ([[150, 150], [370, 222], [740, 444], [1024, 615]] as [$tw, $th]) {
    make_webp_variant("$dir/$name.webp", "$dir/$name-{$tw}x{$th}.webp", $tw, $th);
}

```

9.2 PHP com Imagick (ImageMagick)

Mais rápido e com melhor qualidade de redução do que o GD, mas exige a extensão `imagick` no servidor (nem todos os alojamentos partilhados a têm). Mantém a cor de perfis ICC e suporta mais formatos de entrada (HEIC, TIFF).

```

<?php
function compress_to_webp_imagick(string $src, string $dst, int $maxWidth = 1920, int $targetKb =
350): void
{
    $img = new Imagick($src);

```

```

// Rotação pelo EXIF
switch ($img->getImageOrientation()) {
    case Imagick::ORIENTATION_BOTTOMRIGHT: $img->rotateImage('#000', 180); break;
    case Imagick::ORIENTATION_RIGHTTOP: $img->rotateImage('#000', 90); break;
    case Imagick::ORIENTATION_LEFTBOTTOM: $img->rotateImage('#000', -90); break;
}
$img->setImageOrientation(Imagick::ORIENTATION_TOPLEFT);
$img->stripImage(); // remove EXIF/GPS

if ($img->getImageWidth() > $maxWidth) {
    $img->resizeImage($maxWidth, 0, Imagick::FILTER_LANCZOS, 1); // 0 = mantém a proporção
}

$img->setImageFormat('webp');
foreach ([78, 70, 62, 55, 48] as $quality) {
    $img->setImageCompressionQuality($quality);
    $blob = $img->getImageBlob();
    if (strlen($blob) <= $targetKb * 1024) {
        break;
    }
}
file_put_contents($dst, $blob);
$img->clear();
}

```

9.3 Laravel com Intervention Image 3

A forma mais curta num projeto Laravel. Instalar com `composer require intervention/image`. A versão 3 corrige a rotação EXIF automaticamente e usa GD ou Imagick por baixo.

```

<?php
use Intervention\Image\ManagerInterface;

$manager = ImageManager::gd(); // ou ImageManager::imagick()
$image = $manager->read($request->file('foto'));
$image->scaleDown(width: 1920); // reduz, nunca aumenta

foreach ([78, 70, 62, 55, 48] as $quality) {
    $encoded = $image->toWebp(quality: $quality);
    if (strlen($encoded->toString()) <= 350 * 1024) {
        break;
    }
}

$encoded->save(public_path("uploads/{$name}.webp"));

// Variante recortada ao centro
$manager->read(public_path("uploads/{$name}.webp"))
->cover(370, 222)
->toWebp(quality: 78)
->save(public_path("uploads/{$name}-370x222.webp"));

```

10 Implementação noutras linguagens

A lógica é sempre a mesma: **rodar pelo EXIF** → **reduzir** → **remover metadados** → **codificar em WebP** **baixando a qualidade até ao alvo** → **gerar variantes recortadas**. Muda só a biblioteca. As que usam **libvips** (sharp, govips, image_processing) são as mais rápidas e as que gastam menos memória, porque processam a imagem por partes em vez de a carregar inteira.

10.1 Node.js (sharp)

`npm install sharp`. É a biblioteca de referência em Node, baseada em libvips.

```
const sharp = require('sharp');
const fs = require('fs/promises');

async function compressToWebp(src, dst, maxWidth = 1920, targetKb = 350) {
  // .rotate() sem argumentos corrige a orientação pelo EXIF; os metadados são removidos por omissão
  const base = sharp(src).rotate().resize({ width: maxWidth, withoutEnlargement: true });
  let buf;
  for (const quality of [78, 70, 62, 55, 48]) {
    buf = await base.clone().webp({ quality }).toBuffer();
    if (buf.length <= targetKb * 1024) break;
  }
  await fs.writeFile(dst, buf);
}

async function makeVariant(src, dst, width, height) {
  await sharp(src).resize(width, height, { fit: 'cover', position: 'centre' }).webp({ quality: 78 }).toFile(dst);
}
```

10.2 Python (Pillow)

`pip install Pillow`. Funciona em Django, Flask, FastAPI ou scripts.

```
import io
from PIL import Image, ImageOps

def compress_to_webp(src, dst, max_width=1920, target_kb=350):
    img = ImageOps.exif_transpose(Image.open(src)) # corrige a rotação
    if img.width > max_width:
        img = img.resize((max_width, round(img.height * max_width / img.width)), Image.LANCZOS)
    if img.mode not in ("RGB", "RGBA"):
        img = img.convert("RGBA")
    for quality in (78, 70, 62, 55, 48):
        buf = io.BytesIO()
        img.save(buf, "WEBP", quality=quality, method=6) # sem exif: metadados não são copiados
        if buf.tell() <= target_kb * 1024:
            break
    with open(dst, "wb") as f:
        f.write(buf.getvalue())

def make_variant(src, dst, width, height):
    img = Image.open(src)
```

```
ImageOps.fit(img, (width, height), Image.LANCZOS).save(dst, "WEBP", quality=78) # recorte ao centro
```

10.3 C# / .NET (ImageSharp 3)

`dotnet add package SixLabors.ImageSharp` . Atenção à licença: gratuita para open source e empresas pequenas; empresas maiores precisam de licença comercial.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;
using SixLabors.ImageSharp.Formats.Webp;

static void CompressToWebp(string src, string dst, int maxWidth = 1920, int targetKb = 350)
{
    using var image = Image.Load(src);
    image.Mutate(x => x.AutoOrient()); // rotação pelo EXIF
    if (image.Width > maxWidth)
        image.Mutate(x => x.Resize(maxWidth, 0)); // 0 = mantém a proporção
    image.Metadata.ExifProfile = null; // remove EXIF/GPS

    byte[] bytes = Array.Empty<byte>();
    foreach (var quality in new[] { 78, 70, 62, 55, 48 })
    {
        using var ms = new MemoryStream();
        image.Save(ms, new WebpEncoder { Quality = quality });
        bytes = ms.ToArray();
        if (bytes.Length <= targetKb * 1024) break;
    }
    File.WriteAllBytes(dst, bytes);
}

static void MakeVariant(string src, string dst, int width, int height)
{
    using var image = Image.Load(src);
    image.Mutate(x => x.Resize(new ResizeOptions { Size = new Size(width, height), Mode =
    ResizeMode.Crop }));
    image.Save(dst, new WebpEncoder { Quality = 78 });
}
```

10.4 Go (govips, libvips)

`go get github.com/davidbyttow/govips/v2` . Precisa da libvips instalada no servidor (`apt install libvips-dev`). A biblioteca padrão do Go só lê WebP, não escreve.

```
import (
    "os"
    "github.com/davidbyttow/govips/v2/vips"
)

func CompressToWebp(src, dst string, maxWidth, targetKb int) error {
    img, err := vips.NewImageFromFile(src)
    if err != nil {
        return err
    }
    defer img.Close()
```

```

img.AutoRotate() // rotação pelo EXIF
if img.Width() > maxWidth {
    img.Resize(float64(maxWidth)/float64(img.Width()), vips.KernelLanczos3)
}

var buf []byte
for _, q := range []int{78, 70, 62, 55, 48} {
    p := vips.NewWebpExportParams()
    p.Quality = q
    p.StripMetadata = true
    buf, _, err = img.ExportWebp(p)
    if err != nil {
        return err
    }
    if len(buf) <= targetKb*1024 {
        break
    }
}
return os.WriteFile(dst, buf, 0644)
}

// No arranque da aplicação: vips.Startup(nil); defer vips.Shutdown()

```

10.5 Ruby / Rails (image_processing + libvips)

Já vem com o Active Storage do Rails. `gem "image_processing"` e `libvips` no servidor. A rotação EXIF é aplicada automaticamente.

```

require "image_processing/vips"

ImageProcessing::Vips
  .source(src)
  .resize_to_limit(1920, nil) # reduz só se for maior
  .convert("webp")
  .saver(quality: 78, strip: true)
  .call(destination: dst)

# Variante recortada ao centro
ImageProcessing::Vips.source(dst).resize_to_fill(370, 222).convert("webp").saver(quality:
78).call(destination: thumb)

# Active Storage (Rails 7+): variante WebP direto na view
# <%= image_tag post.cover.variant(resize_to_limit: [1920, nil], format: :webp, saver: { quality:
78, strip: true }) %>

```

10.6 Java (chamar libvips ou ImageMagick)

O ImageIO do Java não escreve WebP. A solução mais simples e robusta é chamar a ferramenta de linha de comando (secção 10.7) a partir do código. A alternativa é a biblioteca `webp-imageio`, que traz binários nativos.

```

import java.io.IOException;

public static void compressToWebp(String src, String dst) throws IOException, InterruptedException
{
    // vips thumbnail: roda pelo EXIF, reduz para 1920 px de largura (só se for maior) e remove

```

```

metadados
    Process p = new ProcessBuilder(
        "vips", "thumbnail", src, dst + "[Q=78,strip]", "1920", "--size", "down")
        .redirectErrorStream(true)
        .start();
    if (p.waitFor() != 0) {
        throw new IOException("vips falhou: " + new String(p.getInputStream().readAllBytes()));
    }
}

```

10.7 Linha de comando (qualquer linguagem, scripts, lotes)

Útil para converter imagens antigas em lote, ou para linguagens sem biblioteca boa. Três ferramentas, todas gratuitas:

```

# ImageMagick 7: roda, remove metadados, reduz só se maior, e pede ao codificador um tamanho-alvo
magick foto.jpg -auto-orient -strip -resize "1920x>" -quality 78 \
    -define webp:target-size=358400 foto.webp

# libvips: o mais rápido e o que gasta menos memória
vips thumbnail foto.jpg "foto.webp[Q=78,strip]" 1920 --size down

# cwebp (Google): atenção, NÃO corrige a rotação EXIF
cwebp -q 78 -resize 1920 0 -metadata none foto.jpg -o foto.webp

# Converter uma pasta inteira (bash)
shopt -s globstar
for f in uploads/**/*.*; do vips thumbnail "$f" "${f%.jpg}.webp[Q=78,strip]" 1920 --size down;
done

```

Dica: o WebP tem um modo "tamanho-alvo" no próprio codificador (`-define webp:target-size` no ImageMagick, `-size` no cwebp). Faz em várias passagens internas o mesmo que o ciclo de qualidades deste guia.

11 Compressão no browser antes do upload

Em ligações lentas, o maior custo para quem publica é **enviar** a foto de 5 MB. O browser pode reduzir e converter a imagem antes do envio: o utilizador envia cerca de 300 KB em vez de 5 MB, e o upload fica 15 vezes mais rápido.

```
async function compressBeforeUpload(file, maxWidth = 1920, quality = 0.78) {
  // imageOrientation: 'from-image' aplica a rotação EXIF
  const bmp = await createImageBitmap(file, { imageOrientation: 'from-image' });
  const scale = Math.min(1, maxWidth / bmp.width);
  const canvas = document.createElement('canvas');
  canvas.width = Math.round(bmp.width * scale);
  canvas.height = Math.round(bmp.height * scale);
  canvas.getContext('2d').drawImage(bmp, 0, 0, canvas.width, canvas.height);

  let blob = await new Promise(r => canvas.toBlob(r, 'image/webp', quality));
  if (!blob || blob.type !== 'image/webp') {
    // Safari antigo não gera WebP: cai para JPEG
    blob = await new Promise(r => canvas.toBlob(r, 'image/jpeg', quality));
  }
  return new File([blob], file.name.replace(/\.w+$/, blob.type === 'image/webp' ? '.webp' : '.jpg'), { type: blob.type });
}

// Uso num formulário
document.querySelector('#foto').addEventListener('change', async (e) => {
  const original = e.target.files[0];
  const leve = await compressBeforeUpload(original);
  const form = new FormData();
  form.append('foto', leve);
  await fetch('/upload', { method: 'POST', body: form });
});
```

Não substitui a compressão no servidor. O código do browser pode ser contornado e nem todos os dispositivos o executam bem. O servidor deve continuar a validar e a comprimir tudo o que recebe; a compressão no browser serve só para poupar dados e tempo a quem envia.

12 Servir as imagens

12.1 Usar a variante certa com srcset

Com as variantes geradas, o HTML diz ao browser quais existem e ele escolhe a mais pequena que serve para aquele ecrã:

```

```

```
loading="lazy" decoding="async"
alt="Descrição da imagem para leitores de ecrã">
```

- `width` e `height` reservam o espaço e evitam que o texto salte quando a imagem carrega.
- `loading="lazy"` só carrega a imagem quando está perto de aparecer no ecrã. Não usar na imagem principal do topo da página.
- `alt` é obrigatório para acessibilidade e ajuda o SEO.

12.2 WebP para imagens antigas sem mudar os URLs

Imagens já publicadas estão referenciadas no HTML dos artigos, por isso não se pode mudar o nome delas. Cria-se uma cópia ao lado (`foto.jpg` → `foto.jpg.webp`) e o servidor entrega-a no mesmo URL aos browsers que enviam `Accept: image/webp` . Os browsers antigos continuam a receber o JPG. No Portal, as cópias são criadas pelo botão **Admin** → **Otimizar Imagens**.

Apache / LiteSpeed – uploads/.htaccess

```
RewriteEngine On
RewriteCond %{HTTP_ACCEPT} image/webp
RewriteCond %{REQUEST_FILENAME}.webp -f
RewriteRule ^(.+)\.(jpe?g|png|gif)$ $1.$2.webp [T=image/webp,L]

<FilesMatch "\.(jpe?g|png|gif)$">
    Header append Vary Accept
</FilesMatch>
```

nginx – dentro do bloco server

```
# O "if" tem de ficar ao nível do server. Dentro de um location, o nginx ignora o try_files.
set $webp_suffix "";
if ($http_accept ~* "image/webp") {
    set $webp_suffix ".webp";
}

location ~* ^/uploads/.\.(jpe?g|png|gif)$ {
    add_header Vary Accept;
    expires 1y;
    try_files $uri$webp_suffix $uri =404;
}
```

Para confirmar: `curl -sI -H "Accept: image/webp" https://site/uploads/foto.jpg` tem de responder `content-type: image/webp` . O cabeçalho `Vary: Accept` é essencial: diz às caches (Cloudflare, Varnish) que a resposta muda conforme o browser, para não servirem WebP a quem não o suporta.

13 Requisitos do servidor

REQUISITO	COMO CONFIRMAR / VALOR	PORQUÊ
GD com WebP (PHP)	<code>php -r 'var_dump(gd_info()["WebP Support"]);'</code> → <code>true</code>	sem isto não existe <code>imagewebp()</code>
Extensão exif (PHP)	<code>php -m grep exif</code>	corrigir a rotação das fotos
<code>memory_limit</code>	256M	uma foto de 12 MP ocupa ~49 MB em RAM
<code>upload_max_filesize</code> / <code>post_max_size</code>	10M / 12M	acima do limite da aplicação (5 MB)
<code>max_execution_time</code>	60 s ou mais	processamento de imagens grandes e lotes
nginx <code>client_max_body_size</code>	20m	senão o nginx rejeita com erro 413 antes de chegar à aplicação
Apache <code>LimitRequestBody</code>	sem limite ou ≥ 20 MB	o mesmo, no Apache
libvips (Node/Go/Ruby/CLI)	<code>vips --version</code>	sharp já a traz incluída; os outros precisam de <code>apt install libvips-tools</code>

14 Cuidados e erros comuns

- **Fotos de lado.** Os telemóveis guardam as fotos "ao alto" deitadas, com uma etiqueta EXIF a dizer para as rodar. Bibliotecas que ignoram o EXIF (GD, cwebp) deixam a foto de lado. Testar sempre com uma foto tirada ao alto.
- **GIF animado.** O GD guarda só o primeiro frame. Deixar como está, ou usar sharp/libvips/ImageMagick, que geram WebP animado.
- **Fundo preto em PNG.** Ao converter imagens com transparência no GD, chamar `imagepalettetotruecolor` e `imagesavealpha(..., true)`.
- **Cores estranhas.** Fotos de câmaras profissionais podem vir em CMYK ou com perfis ICC. Imageick e libvips tratam disto; o GD pode alterar as cores.
- **Formato HEIC do iPhone.** O GD não o lê. O iPhone converte para JPEG ao enviar pelo browser na maioria dos casos; para HEIC direto, usar Imageick ou libvips com suporte HEIF.
- **Original descartado.** O Portal apaga a foto original para poupar disco. Se o projeto precisar dela (impressão, por exemplo), guardar o original numa pasta fora do webroot.
- **Nunca aumentar.** Não ampliar imagens pequenas nem recomprimir acima do tamanho atual.
- **Compressão repetida.** Cada vez que uma imagem com perdas é recodificada, perde um pouco de qualidade. Codificar a partir do original, uma vez.

- **Nomes de ficheiros.** Guardar com nomes ASCII sem espaços (`1790874292-foto.webp`). Acentos nos nomes causam URLs partidos em alguns servidores.

15 Checklist de implementação

1. Confirmar no servidor a biblioteca de imagem com WebP e o suporte EXIF.
2. Ajustar os limites: `memory_limit` , upload e o limite do servidor web (nginx/Apache).
3. Decidir a largura máxima, o alvo em KB e as variantes a partir do layout.
4. Validar o tamanho no browser antes de enviar (opcional: comprimir no browser).
5. No servidor, chamar a função de compressão logo depois de receber o upload.
6. Gerar só as variantes que o layout usa, com recorte centrado.
7. Nas páginas, usar `srcset` , `width / height` e `loading="lazy"` .
8. Para imagens antigas: gerar cópias `.webp` e configurar o servidor com `Vary: Accept` .
9. Testar com fotos reais de telemóvel: ao alto, deitada, PNG com transparência e GIF animado.
10. Medir: comparar o tamanho antes e depois, e confirmar `content-type: image/webp` com `curl` .



O Portal do Mundo é um portal de notícias e histórias em português, feito em Moçambique, onde os leitores apoiam diretamente quem escreve. Este livro documenta uma das funcionalidades que o tornam leve para quem lê pelo telemóvel.

www.portaldomundo.com

© 2026 Humberto Rodrigues · Portal do Mundo